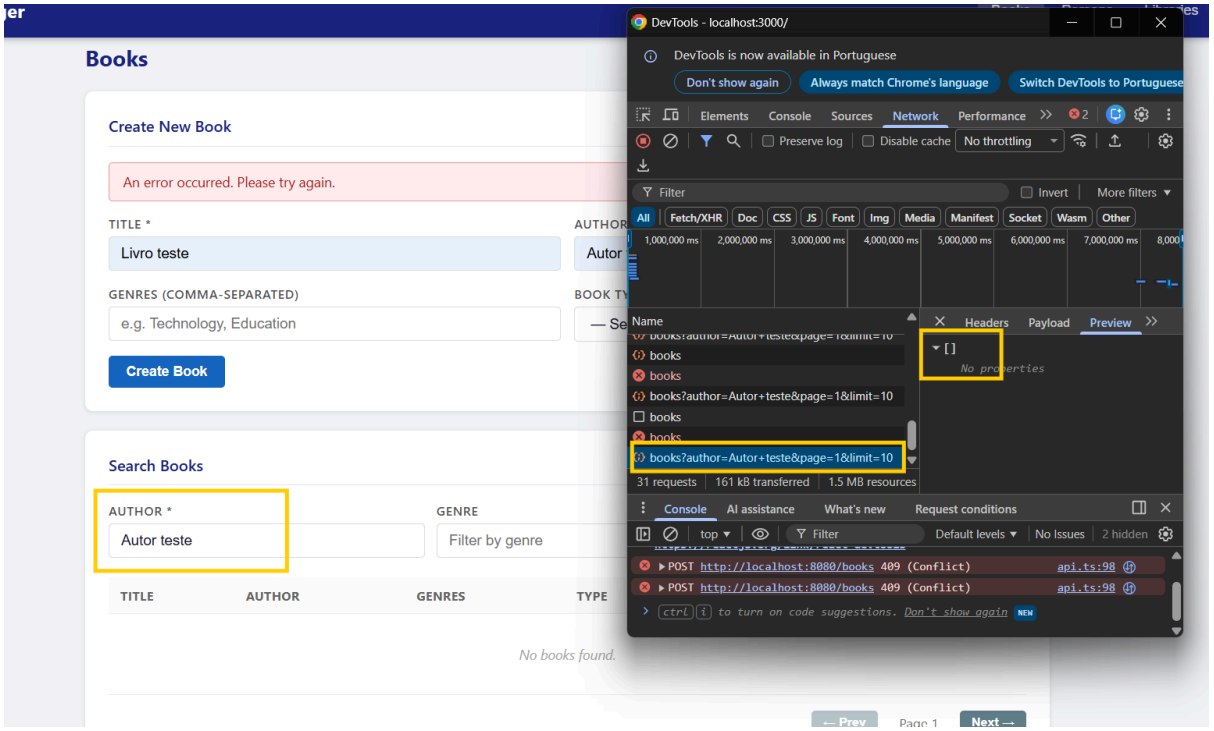
Relatório de Incidente de Funcionalidade (Search Engine) - Goledger Challenge

ID do Chamado: BUG-004 | **Prioridade:** Alta (Crítica para Operação) **Módulo:** Busca e Filtragem (Front-end / Back-end) **Plataforma:** Aplicação Web e API **Relator:** Vitória (QA)




```
export async function getBooks(params: GetBooksParams): Promise<Book[]> {
  const query = new URLSearchParams({
    author: params.author,
    page:   String(params.page),
    limit:  String(params.limit),
  });

  if (params.genre) {
    query.set('genre', params.genre);
  }

  const res = await fetch(`${API_BASE}/books?${query}`, {
    headers: { Authorization: `Bearer ${getToken()}` },
```

1. Descrição do Problema Identificou-se uma falha crítica no mecanismo de busca por autores. Após a criação e confirmação de persistência de um novo livro, ao realizar a consulta utilizando o nome exato do autor no campo de busca, o sistema retorna um resultado vazio. A análise técnica via *Network DevTools* mostra que a API responde com status `200 OK`, porém o corpo da resposta (`body`) entrega um array vazio, evidenciando uma falha na lógica de filtragem ou na indexação dos dados no Back-end.

2. Passos para Reproduzir (Reproducibility)

- **Criação:** Cadastre um novo livro (Ex: Título: "Ensaio sobre a Cegueira", Autor: "Saramago").
- **Confirmação:** Certifique-se de que o livro aparece na listagem geral após o cadastro.
- **Busca:** No campo de pesquisa, digite exatamente o nome do autor cadastrado ("Saramago") e acione o filtro.
- **Observação Visual:** Note que a tabela de resultados fica vazia ou exibe a mensagem "Nenhum resultado encontrado".
- **Análise de Rede:** Verifique que a requisição `GET` para o endpoint de busca retorna `200 OK` com o payload `[]`.

3. Análise da Causa Raiz (Evidência em Anexo) Conforme detalhado no **print do código-fonte anexado a este reporte**, a falha reside na construção do objeto `URLSearchParams` dentro da função `getBooks`.

Diferente do parâmetro `genre` (que possui um tratamento condicional logo abaixo no código), os campos destacados na imagem — especificamente o `author` — estão sendo injetados de forma estática e obrigatória na URL. Isso faz com que a API receba uma string de busca mesmo quando o usuário não digitou nada, resultando em um filtro por valor vazio que anula os resultados legítimos do banco de dados.

-Solução feita para a continuação dos testes na plataforma:

```go
var finalBooks []interface{}

if genre != "" {
    for _, book := range books {
        bookMap, ok := book.(map[string]interface{})
        if !ok {
            continue
        }
        genres, ok := bookMap["genres"].([]interface{})
        if !ok {
            continue
        }
        for _, g := range genres {
            if gs, ok := g.(string); ok && gs == genre {
                finalBooks = append(finalBooks, book)
                break
            }
        }
    }

} else {
    finalBooks = books
}
```

4. Impacto Operacional

- **Localização de Ativos:** Impede que bibliotecários e usuários encontrem livros específicos em acervos grandes, tornando a funcionalidade de busca inútil.
- **Integridade Percebida:** Passa a falsa impressão de que os dados não foram salvos, gerando retrabalho e duplicidade de cadastros por parte do usuário.

5. Recomendações Técnicas e Solução

- **Normalização de Busca:** Implementar busca *case-insensitive* e remoção de espaços em branco (trim) tanto no Front quanto no Back-end.
- **Revisão de Contrato:** Validar se o nome do campo enviado pelo formulário de busca coincide exatamente com o que a API espera receber no endpoint de consulta.
- **Logs de Consulta:** Revisar os logs do servidor para verificar se a query disparada ao banco de dados está filtrando pelo campo correto (`author`).

6. Evidência de Fechamento (Expectativa Pós-Correção) Ao pesquisar por um autor existente, a API deve retornar o status `200 OK` acompanhado do objeto ou lista de objetos correspondentes no JSON, e o Front-end deve renderizar esses itens imediatamente na tabela de resultados.